DBMON-02: SQL Database Monitoring

> **Series:** DBMON — Database Monitoring | **Notebook:** 2 of 7 | **Created:** March 2026 | **Last Updated:** 11/13/2026

Overview

This notebook focuses on monitoring relational SQL databases with Dynatrace. You will learn how to analyze query performance for PostgreSQL, MySQL, Microsoft SQL Server, and Oracle databases. We cover slow query detection, operation-level breakdowns, connection monitoring, and response time analysis using distributed trace spans.

Table of Contents

1. SQL Database Landscape
2. Query Performance Analysis
3. Slow Query Detection
4. Operation Breakdown
5. Connection Pool Monitoring
6. Database-Specific Patterns
7. Response Time Distribution
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail (Managed has no Grail, so these DQL cells do not run there)
OneAgent	Deployed on application hosts with SQL database clients
Permissions	`storage:spans:read`, `storage:entities:read`
Data	Application traffic generating SQL database calls (PostgreSQL, MySQL, MS SQL, or Oracle)
Prior Reading	DBMON-01: Database Monitoring Fundamentals

1. SQL Database Landscape

Relational databases share a common monitoring model: they all execute SQL statements against tables in schemas. Dynatrace captures these calls consistently across vendors using the OpenTelemetry `db.*` span attributes.

Vendor	`db.system` Value	Common Operations	Default Port
PostgreSQL	`postgresql`	`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `COPY`	5432
MySQL	`mysql`	`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CALL`	3306
MariaDB	`mariadb`	`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `CALL`	3306
Microsoft SQL Server	`mssql`	`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `EXEC`	1433
Oracle	`oracle`	`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `MERGE`	1521
IBM Db2	`db2`	`SELECT`, `INSERT`, `UPDATE`, `DELETE`	50000

> **A call is not always a statement, and the operation name is not always the statement kind.** On OneAgent-captured SQL spans, `db.operation.name` holds the leading keyword of what the driver sent, and also records connection and driver phases. On the validation tenant (SQL Server, 1 h, 10/02/2026): `SELECT` 20,820 · `CONNECT` 16,723 · `RESULTSET` 1,112 · `SET` 1,089 · `EXECUTE` 1,015 · `COMMIT` 964 · `PREPARE` 580 · `INSERT` / `UPDATE` / `DELETE` 11.
>
> - `CONNECT` and `COMMIT` carry no query text: they open a connection and end a transaction.
> - `PREPARE` and `RESULTSET` repeat a statement's text. Every one of them shared its trace and its `db.query.text` with a `SELECT`, `SET` or write span, so a cell keyed on the text counts that statement twice unless it drops them.
> - Batched writes are named after their first keyword. 974 of the 1,089 `SET` spans were `SET NOCOUNT ON; INSERT …` batches, so a read/write split on `db.operation.name` alone would report almost no writes. § 4 classifies from the statement text instead.
>
> Cells that measure queries drop the four non-statement operations (DBMON-01 § 4). Cells that count calls or failures keep them.
>
> **Dictionary:** `db.system` (`experimental`) — `mysql` and `mariadb` are separate `supported_values`, read 10/23/2026.

Let's start by identifying which SQL databases are active in your environment.

```
// Field names corrected 08/12/2026 — pre-1.0 OpenTelemetry database semconv
names had been
// used throughout, and none of them has a row in the semantic dictionary
(older OTel
// instrumentations may still emit db.statement). They fail SILENTLY: a filter
on a
// non-existent field matches nothing and a summarize groups everything under
null, so these cells
// returned empty or single-null-group results without ever erroring.
//   db.operation         -&gt; db.operation.name    (stable; set on 50,379 of
57,295 db spans)
//   db.statement         -&gt; db.query.text        (stable; 33,463)
//   db.mongodb.collection-&gt; db.collection.name   (stable; 6,912)
//   db.name              -&gt; db.namespace         (stable; 57,281)
// Confirm the catalog for your tenant with:
//   fetch dt.semantic_dictionary.fields | filter startsWith(name, "db.") |
fields name, stability
// Discover active SQL databases in the environment
fetch spans, from:-1h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| summarize {
    call_count = count(),
    avg_duration_ms = avg(duration) / 1ms,
    p95_duration_ms = percentile(duration, 95) / 1ms,
    unique_queries = countDistinct(db.query.text)
}, by:{db.system, db.namespace, server.address}
| sort call_count desc
```

2. Query Performance Analysis

The most important aspect of SQL database monitoring is understanding query performance. Parameterized statements keep their placeholders and `WHERE`-clause literals are masked (DBMON-01 § 1), so `db.query.text` groups identical query shapes together.

```
// Top 20 SQL queries by total execution time (highest impact)
fetch spans, from:-1h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| filter isNotNull(db.query.text)
| summarize {
    total_time_ms = sum(duration) / 1ms,
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms
}, by:{db.system, db.query.text}
| sort total_time_ms desc
| limit 20
```

```
// Query throughput over time by database system
fetch spans, from:-6h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| makeTimeseries queries_per_min = count(), by:{db.system}, interval:1m
```

3. Slow Query Detection

Slow queries are the most common cause of database-related performance problems. A query is considered "slow" relative to its own baseline or an absolute threshold. We use both approaches below.

```
// dt.service.name: dt.entity.service is deprecated in the semantic dictionary
(dt.service.name is stable).
// Detect slow queries — calls exceeding 500ms
fetch spans, from:-1h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| filter duration &gt; 500ms
| fields start_time, db.system, db.namespace, db.operation.name,
        db.query.text, server.address,
        duration_ms = duration / 1ms,
        dt.service.name
| sort duration_ms desc
| limit 25
```

```
// Slow query frequency over time — how often do queries exceed 500ms?
fetch spans, from:-6h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| makeTimeseries {
    total = count(),
    slow = countIf(duration &gt; 500ms)
  }, interval:10m
```

```
// Identify query patterns with the highest P95 — potential optimization
candidates
fetch spans, from:-1h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| filter isNotNull(db.query.text)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    max_ms = max(duration) / 1ms
}, by:{db.query.text, db.system}
| filter call_count &gt;= 10
| sort p95_ms desc
| limit 15
```

4. Operation Breakdown

Understanding the mix of reads and writes shows where load comes from. Grouping by table needs `db.collection.name`, which was not set on any SQL span on the validation tenant, so the breakdown below is by database and operation type.

```
// Operation mix by database — reads, writes and everything else, classified from the statement text
// db.operation.name is only the leading keyword: an ORM batch "SET NOCOUNT ON; INSERT …" is named SET.
// PREPARE and RESULTSET repeat a statement's text, so they are dropped to avoid counting it twice.
// The text test is approximate: a CTE that writes (WITH … INSERT) is counted as READ.
fetch spans, from:-1h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle", "db2"})
| filter isNotNull(db.query.text)
| filter not(in(coalesce(db.operation.name, ""), {"PREPARE", "RESULTSET"}))
| fieldsAdd q = upper(trim(db.query.text))
| fieldsAdd op_type = if(startsWith(q, "SELECT") or startsWith(q, "WITH"), then:"READ",
    else:if(contains(q, "INSERT ") or contains(q, "UPDATE ") or contains(q, "DELETE ") or contains(q, "MERGE "), then:"WRITE",
    else:"OTHER"))
| summarize op_count = count(), by:{db.system, db.namespace, op_type}
| sort db.system asc, op_count desc
```

```
// Average response time by operation type — are writes slower than reads?
fetch spans, from:-1h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle", "db2"})
| filter isNotNull(db.operation.name)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms
}, by:{db.operation.name, db.system}
| sort db.system asc, avg_ms desc
```

5. Connection Pool Monitoring

Connection pool exhaustion is a common source of application errors. While Dynatrace does not directly expose connection pool counters through spans, you can infer connection pressure by analyzing database call volume per service and error patterns (call counts are not concurrency — pool counters come from runtime metrics, see DBMON-05 § 5).

```
// dt.service.name: dt.entity.service is deprecated in the semantic dictionary
(dt.service.name is stable).
// Database calls per service — identify which services make the most DB calls
fetch spans, from:-1h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    error_count = countIf(span.status_code == "error")
}, by:{dt.service.name, db.system, server.address}
| sort call_count desc
| limit 20
```

```
// Database error patterns — connection refused, timeout, deadlock
// span.status_message is experimental and is often empty on failed DB spans;
the error
// text is recorded as an exception event in span.events. expand counts
exception events,
// so a span with two exception events counts twice.
fetch spans, from:-6h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| filter span.status_code == "error"
| expand span.events
| filter isNotNull(span.events[exception.type])
| fieldsAdd exception_type = span.events[exception.type], exception_message =
span.events[exception.message]
| summarize error_count = count(), by:{db.system, server.address,
exception_type, exception_message}
| sort error_count desc
| limit 20
```

6. Database-Specific Patterns

Each SQL database has unique characteristics worth monitoring. The following queries target vendor-specific patterns.

PostgreSQL

PostgreSQL is commonly used in cloud-native applications. Key concerns: vacuum operations, index bloat, and lock contention.

```
// PostgreSQL — query performance breakdown by database
fetch spans, from:-1h
| filter db.system == "postgresql"
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    errors = countIf(span.status_code == "error")
}, by:{db.namespace, db.operation.name}
| sort call_count desc
```

MySQL / MariaDB

MySQL monitoring focuses on InnoDB buffer pool usage and replication lag. Query cache effectiveness applies only before MySQL 8.0, which removed the query cache (What Is New in MySQL 8.0 (MySQL docs): *"The query cache was removed."*).

```
// MySQL / MariaDB — response time trend over 6 hours
fetch spans, from:-6h
| filter in(db.system, {"mysql", "mariadb"})
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| makeTimeseries {
    avg_ms = avg(duration / 1ms),
    p95_ms = percentile(duration / 1ms, 95),
    call_count = count()
  }, interval:10m
```

Microsoft SQL Server — Server-Side View via the ActiveGate Extension

Everything above measures SQL Server from the **caller side** — spans emitted by your instrumented applications (`db.system == "mssql"`). That view cannot see engine internals: blocking, transaction-log pressure, Always On replica health, agent job failures, or databases no instrumented service talks to. The **Microsoft SQL Server extension** (Dynatrace Hub, runs remotely on an ActiveGate — no agent on the DB host) completes the picture with documented `sql-server.*` metrics and job-outcome log streams:

Feature set	Key signals
Default (always on)	`sql-server.general.processesBlocked`, `sql-server.databases.state`, `sql-server.uptime`, user connections, memory/CPU/worker threads
Transaction Logs	`sql-server.databases.log.percentUsed`, used/total size, growth/shrink/flush-wait counts
Database Files	Per-file size / used / empty space + `largest_files` log stream
Always On	`.ag.synchronizationHealth`, `.ar.role` / `.ar.failoverMode` / `.ar.operationalState`, `.db.synchronizationState`, log send/redo queues
Jobs + **Agent**	`current_jobs` / `failed_jobs` log streams (outcome, duration, **error message text**) + `sql-server.sql.agent.status`

Deployment notes that matter:

- **Always On clusters need two monitoring configurations** — the Always On feature set pointed at **primary replicas only**, everything else at all instances. Mixing primaries and secondaries in one Always On config duplicates metrics (Microsoft SQL Server extension (DT docs): *"Don't enable the "Always On" feature set on both the primary and the secondary replica in the same cluster."*).
- **Job outcomes arrive as log streams, not metrics** — alert with a DQL log alert or an OpenPipeline log-to-metric rule; the failure text is queryable in Grail.
- Logs from official database extensions land in the dedicated `default_database_monitoring` Grail bucket (see DBMON-01 §6).
- **Minimum ActiveGate version 1.303.** The extension docs are explicit: *"If you're not on ActiveGate 1.303 and newer, your monitoring configurations will error upon running."*

This is a hard-failure prerequisite, not a recommendation — check the ActiveGate group's version before creating the monitoring configuration.

- The extension ships a fixed set of predefined queries; **the documentation does not describe a user-defined-SQL option**, so treat custom SQL as unavailable unless your extension version's page says otherwise. True gaps (e.g., tempdb version-store pressure) go in a small custom Extensions 2.0 `sqlServer`-datasource extension on the same ActiveGate.

For the full decision framework — mapping a homegrown script/Telegraf monitor estate onto the extension, the honest gaps, and the migration sequence — see **FAQ-14: Should I Replace My Custom SQL Server Monitoring Scripts with the Dynatrace Extension?**

Blocked processes from the engine's point of view — the signal caller-side spans can only infer from slow response times:

> **Sources:** Microsoft SQL Server extension (DT docs) — feature sets, the `sql-server.*` metric keys, the jobs-as-log-streams model, the Always On two-configuration rule and the ActiveGate 1.303 minimum, Microsoft SQL Server local extension (DT docs) — the OneAgent-host variant, Extensions (DT docs) — the framework these run on. Read at source 08/27/2026.

```
// SQL Server engine blocking — from the ActiveGate extension (Default feature
set)
//
// An empty result means the SQL Server extension is not deployed in this
// environment — NOT a wrong key and not a broken query. `sql-server.*` keys
// only exist once a monitoring configuration is running. Separate the two
// before debugging: `metrics | filter startsWith(metric.key, "sql-server")`
// returning 0 while `startsWith(metric.key, "dt.host.")` returns rows means
// the discovery surface works and the extension simply is not there.
// (`metrics` takes `from:` with NO leading comma, and caps discovery at 10
days.)
timeseries blocked = avg(`sql-server.general.processesBlocked`), from:-24h
```

```
// Always On availability-group sync health — extension Always On feature set
// (point the Always On monitoring configuration at the primary replica)
//
// An empty result means the SQL Server extension is not deployed in this
// environment — NOT a wrong key and not a broken query. `sql-server.*` keys
// only exist once a monitoring configuration is running. Separate the two
// before debugging: `metrics | filter startsWith(metric.key, "sql-server")`
// returning 0 while `startsWith(metric.key, "dt.host.")` returns rows means
// the discovery surface works and the extension simply is not there.
// (`metrics` takes `from:` with NO leading comma, and caps discovery at 10
days.)
timeseries agHealth = min(`sql-server.always-on.ag.synchronizationHealth`),
from:-24h
```

7. Response Time Distribution

Understanding the distribution of response times helps set realistic SLOs and identify bimodal patterns (e.g., cached vs uncached queries).

```
// Response time distribution buckets — group queries into latency tiers
// The numeric prefix makes the tiers sort in latency order rather than
alphabetically.
fetch spans, from:-1h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| fieldsAdd duration_ms = duration / 1ms
| fieldsAdd latency_tier = if(duration_ms &lt; 1, then:"1: &lt;1ms",
    else:if(duration_ms &lt; 10, then:"2: 1-10ms",
    else:if(duration_ms &lt; 100, then:"3: 10-100ms",
    else:if(duration_ms &lt; 1000, then:"4: 100ms-1s",
    else:"5: &gt;1s"))))
| summarize query_count = count(), by:{latency_tier}
| sort latency_tier asc
```

```
// Percentile summary across all SQL databases
fetch spans, from:-1h
| filter in(db.system, {"postgresql", "mysql", "mariadb", "mssql", "oracle",
"db2"})
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize {
    p50_ms = percentile(duration, 50) / 1ms,
    p90_ms = percentile(duration, 90) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    p99_ms = percentile(duration, 99) / 1ms,
    max_ms = max(duration) / 1ms,
    total_calls = count()
}, by:{db.system}
| sort total_calls desc
```

8. Summary and Next Steps

In this notebook you learned:

- How to identify and inventory SQL databases in your environment using span data
- Techniques for finding the highest-impact and slowest queries
- How to separate statements from connection and driver calls, and reads from writes
- Connection pressure and error pattern analysis
- Vendor-specific monitoring patterns for PostgreSQL and MySQL, plus the server-side SQL Server view via the ActiveGate extension (`sql-server.*` metrics, job-outcome log streams)
- Response time distribution analysis for setting realistic SLOs

Next Steps

- **DBMON-03: NoSQL Database Monitoring** — MongoDB, Cassandra, DynamoDB, and Cosmos DB analysis
- **DBMON-05: Query Analysis** — Deep dive into N+1 detection, query frequency patterns, and optimization

Where to Go Deeper

- **AIOPS series** — Davis anomaly detection on DB connection-pool exhaustion, error rate spikes, and slow-query rate anomalies
- **AUTOM series** — Monaco / Terraform automation for vendor-specific extension deployment at scale
- **DBMON-05** — Deep query analysis (N+1 detection, optimization prioritization)
- **FAQ-14** — Should I replace my custom SQL Server monitoring scripts with the Dynatrace extension? (decision framework, gap handling, migration sequence)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.